

Effective Invasive Plant Identification using Convolutional Neural Networks

Authors: Olivia Gette, Zoe Mazurkiewicz, and Erica Zhu
Michigan Technological University

Abstract

Early Detection and Rapid Response (EDRR) plays a critical role in mitigating the ecological and economic damages caused by invasive species. This study aims to identify the most effective convolutional neural network (CNN) model for classifying invasive plant species using image data from a Brazilian national forest. Five models were trained and evaluated using performance metrics like accuracy, precision, recall, F1-score, and confusion matrix: VGG16, DenseNet, MobileNet, InceptionV3, and a custom model. The custom model uses three convolutional layers and two dropout layers, which was designed to be lightweight and reduce overfitting on a smaller, imbalanced dataset. Results showed that the custom model performed the best of the five models, doing very well at correctly identifying invasive species despite having the lowest overall accuracy compared to the pretrained models.

Introduction

The progression of climate change has left no population unscathed with its implications, from more severe and frequent natural disasters to food insecurity. Irregular seasons, as one of the greatest indicators of climate change, provide favorable conditions for invasive species to thrive in non-native areas. While select non-native species remain benign, invasive species disrupt native ecosystems by outcompeting indigenous flora and fauna, erode soil composition, and impose significant environmental and economic costs in their wake.

The need for Early Detection and Rapid Response is important to lessen the impact of invasive species. Traditional methods of identifying invasive plants rely on manual inspection, which is time-consuming and resource-intensive. Advances in image-based machine learning provide a scalable solution by automating species identification and enabling rapid classification of invasive plants from simple photos to multispectral imaging (with robust tools such as the airborne visible/infrared imaging spectrometer, or AVIRIS) [1]

Machine learning methods, including convolutional neural networks, have proven to be effective for image recognition tasks such as medical diagnostics, facial recognition, and environmental monitoring. For instance, CNNs have been employed to detect cancers through radiographic images, classify wildlife species in conservation efforts, and predict crop diseases in agriculture. Their ability to automatically extract features from images makes them good for tasks that have complex visual data.

Prior research related to invasive species detection utilizes pre-trained neural network models to classify plant species. These approaches have demonstrated high accuracy but can face limitations such as overfitting when datasets are small or imbalanced [2]. Custom CNN architectures, on the other hand, can be edited for lightweight applications while addressing overfitting through regularization techniques like dropout layers [3]. While pretrained models offer robust feature extraction, custom models provide flexibility.

This report explores the application of CNNs for invasive species detection, evaluating five models (VGG16, DenseNet, MobileNet, InceptionV3, and a custom model) on image data from a Brazilian national forest. By comparing their strengths and weaknesses, this study aims to identify the most effective model for accurate classification of invasive plants, contributing to the overall goal of ecosystem protection.

Dataset

It is necessary to preface that useful datasets for invasive species were few and far between; Missing and limited information was one of the greatest challenges for finding a usable dataset. We were unable to obtain enough image data for species classification and find substantial documentation of Michigan plant populations. The scope of the project was limited to optimizing invasive species classification from a 2017 Kaggle dataset based in Brazil, *Invasive Species Monitoring* [4].

The dataset contains 1468 training, 368 validation, and 459 test images. The images were initially labeled 0 or 1, with a probability of 1 meaning that the species is present, having 847 labeled as non-invasive (0). We first split the dataset 80-20 for training and testing, and then split the training set 80-20 for a validation set. Due to the small number of samples in the dataset, we felt it necessary to preprocess the training data further to obtain more variability via data augmentation. Analysis of labeled data revealed that 847 images were non-invasive and 1,448 were invasive, meaning there was a need to handle a class imbalance within the data. To do so, we use the `compute_class_weight()` function to assign the non-invasive class and invasive class a weight of 1.3618 and 0.7901, respectively.

The standard normalization for image models is applied to rescale the image from [0, 255] to [0,1] for ease of training our CNNs.

Methods

There are several different algorithms that we used in this project. One method is Convolutional Neural Networks (CNN). Convolutional Neural Networks are a class of deep learning models designed for image recognition tasks. They automatically extract spatial features from input images using a series of convolutional layers, where filters slide over the image to detect patterns like edges, textures, and shapes. After convolutions, pooling layers reduce the spatial dimensions of the feature maps to minimize computational cost and overfitting. Finally, fully connected layers integrate the extracted features and make the final classification.

The architectures used in this project included InceptionV3, VGG16, MobileNet, and DenseNet. These pretrained models use transfer learning, where a model trained on a large dataset is fine-tuned for a specific task. The convolutional layers serve as feature extractors, while custom layers (dropout layers) adapt the model for classifying invasive plants. Transfer learning makes training more efficient and improves performance, especially on smaller datasets. InceptionV3 incorporates multi-scale convolutional filters and optimizes computational efficiency. VGG16 is a deep CNN with a sequential architecture, using fixed-size 3x3 * 3 filters and 2x2 * 2 max-pooling layers. MobileNet is a lightweight model designed for mobile devices, which uses

depth wise separable convolutions to reduce complexity. Finally, DenseNet connects each layer to every other layer, promoting feature reuse and improving gradient flow.

We also used a custom CNN model. It was developed to address a specific challenge we faced in our project, which was the small, imbalanced dataset. The model consists of three convolutional layers with ReLU activation and includes two dropout layers, with a dropout probability of 50%, to reduce the chances of overfitting by randomly deactivating neurons during training. The final fully connected layers map the learned features to a binary classification output using a sigmoid activation. The output represents the probability of an image being classified as an invasive plant.

The models were trained using binary cross-entropy loss:

$$Loss = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad [5]$$

Where y_i is the true label, $\hat{y}_i$ is the predicted probability, and N is the total number of samples. The Adam optimizer was also used to adjust model parameters.

Results and Discussion

To fine tune our model, we chose a couple of hyperparameters. These included: learning rate, dropout rate, classification threshold, batch size, and epoch number. For the learning rate, we chose to use the Adam (Adaptive Moment Estimation) optimizer. The reason we chose this optimizer is because of its adaptive learning rates, bias correction, and computational efficiency. The Adam optimizer adjusts the learning rate for every parameter, which makes it stronger against variations in gradient [6]. We chose a dropout rate of 50% for our custom model since this is a common dropout rate for machine learning models. Dropout rates help prevent overfitting by dropping 50% of the neurons in a layer every iteration. It makes the model less sensitive to specific training examples and promotes generalization by forcing the model to learn more robust and generalizable features. We decided to test two different classification thresholds for this project, 0.4 and 0.7. This allows us to see whether a higher or lower threshold would be best for our overall goal of classifying invasive species as consistently as possible. Our threshold of 0.4 aids recall, while our threshold of 0.7 aids precision. We chose a batch size of 32 because smaller batch sizes can lead to faster convergence and better generalization. Larger batch sizes take less time to train, but risk lower accuracy [7]. Lastly, we chose an epoch number of 20 because all of the models were able to level off and stop improving accuracy with a patience of 5 epochs within 20 total epochs. We also didn't want to risk overfitting our model.

Our biggest challenge during this project was our dataset. As mentioned, we had a small dataset which led us into some issues, one being how poorly our models were performing. We decided it would be best to use data augmentation in hopes of improving our models' performance, which ended up working well. Even though we probably would get better results with a larger, more clean dataset, we were able to work with what we had as well as we could.

Regarding our evaluation metrics, we used confusion matrices, precision, recall, F1-score, and accuracy. For accuracy, we used an overall percentage for each model tested and for the rest of

the metrics, we ran them for the two different thresholds of 0.4 and 0.7 for each model. Confusion matrices give us a breakdown of the model predictions, showing true positives, true negatives, false positives, and false negatives. It also gives us insight into the classification performance. Precision evaluates how many of the predicted positive cases (invasive) are actually correct. Recall examines how many true positive cases are correctly identified by the model. F1-score combines precision and recall into a single score. Finally, accuracy measures the proportion of correct predictions across all samples.

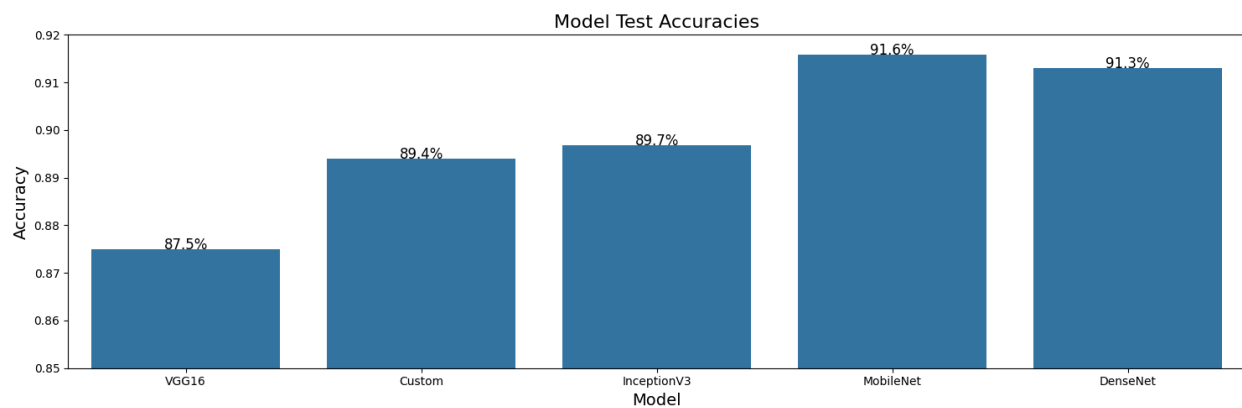


Figure 1: This is a bar graph that visually displays the accuracies of the different models after training and testing. As you can see, the model that had the highest accuracy was the MobileNet model (91.6%), while the model with the lowest accuracy was the VGG16 model (87.5%). Overall, all of our models had pretty good accuracy considering our project focus and our dataset.

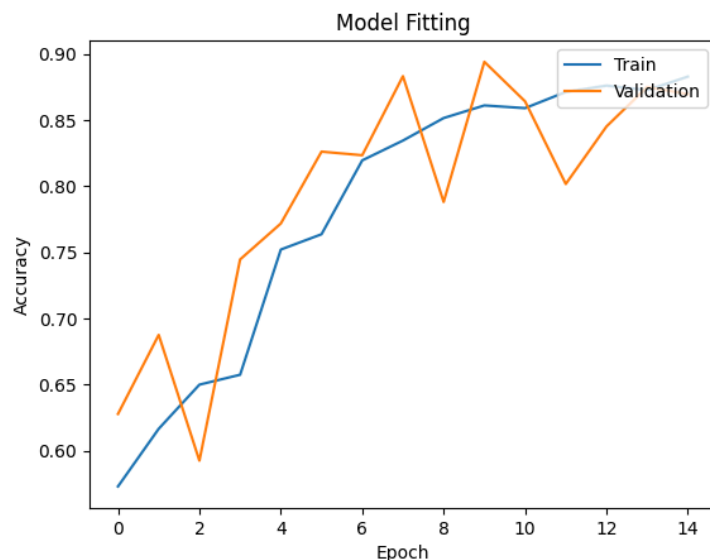


Figure 2: We had a graph like this run while each of the five models were fitting. Above we have the graph that compares accuracy as the epochs increase for the custom model. All five models had the same general shape, where the accuracy increased as the epochs increased. Accuracy increases in early epochs as the model learns basic patterns, but eventually levels off when further training provides less improvements. At this point, the model reaches an optimal fit, where additional training risks overfitting rather than improving generalization.

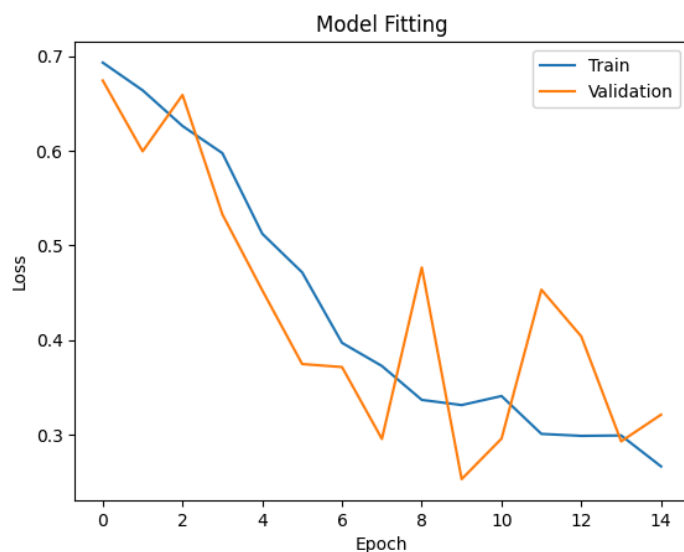


Figure 3: We also had a graph like this run while each of the five models were fitting. In this case, we have the graph that compares loss as the epochs increase for the custom model. Again, all five models had the same general shape, where the loss decreased as the epochs increased. This confirms that we are using a good number of epochs, since they are continually decreasing until the end of the training, where it starts to level out. Loss decreases over epochs because the model continuously adjusts its weights through optimization, learning better features and reducing errors in predictions. As training progresses, the Adam optimizer refines learning rates, ensuring gradual improvement while preventing overshooting.

The results of this project showed that DenseNet was the best-performing model for invasive plant classification, offering the strongest balance across key evaluation metrics. It achieved the second-highest accuracy (91.3%) and had the best F1 score at the 0.4 threshold (0.617), meaning it minimized both false positives and false negatives effectively. Its ability to extract and reuse features efficiently contributed to its strong performance across precision and recall metrics, making it the most reliable choice for accurate invasive species identification.

We also compared two classification thresholds (0.4 and 0.7) and found that the 0.7 threshold significantly reduced recall, meaning more invasive species were missed. While precision slightly improved at 0.7, it did not compensate for the drop in recall, leading to lower F1 scores. As a result, we chose the 0.4 threshold as the best option, which ensures invasive species are detected consistently while maintaining solid precision and classification balance.

Conclusion

The best performing model overall for our purpose of classifying potentially invasive plants was the DenseNet model, with the highest precision and F1 score at a lower threshold of 0.4. The InceptionV3 and VGG16 were the least effective primarily due to complex architecture and overfitting, respectively, which are symptoms that may arise from small or less robust datasets. The more successful qualities in the DenseNet, MobileNet, and Custom models arise from flexible, lightweight architecture that prioritizes recall because a false negative (failing to detect an invasive species at all) is more problematic in our scenario.

Future work would include finding a large image dataset native to Michigan such that our findings might be more applicable locally. Other possible experimentations involve finding the most suitable threshold for a less drastic tradeoff between precision and recall. Expansions of this project would include building a user friendly application, or even predictive mapping with decision boundaries of invasive species using remote sensing tools (drone imaging, etc).

Contributions

Zoe worked on finding the dataset and training a YOLO model with the data. As we could not cross reference its results with the rest of the models, we weren't able to implement it into the project as-is. Zoe is the creator of our project poster. She also contributed significantly to our project presentations. She also retrieved graphs for accuracy and loss of each model.

Olivia worked on the creation and implementation of the Custom CNN, InceptionV3, VGG16, MobileNet, and DenseNet models. This included splitting the data, creating the data generators, performing data augmentation on the training data, computing the class weights, and adding necessary layers so that all the models could be compared. After training, evaluating, and testing the models, she ran different evaluation metrics for all the models, compared the results across two different thresholds, and visualized model accuracies and metrics using bar graphs. Additionally, she developed code to print image examples of model predictions and analyzed the results to determine the optimal model and threshold for the project. She also assisted in preparing the presentation and contributed significantly to the final report's writing.

Erica worked on finding additional datasets, configuration from Kaggle, and unzipping data. She was able to train and produce results using the data in a ResNet18 model, but was not able to implement it into project code (the code was too different, and the model was not necessary by the end). Erica assisted in preparing the presentation and contributed to the final report in writing and synthesizing articles.

References

- [1] Jensen et. al. (2020). “*Employing Machine Learning for Detection of Invasive Species using Sentinel-2 and AVIRIS Data: The Case of Kudzu in the United States.*” Sustainability, Vol 12, Is 9. <https://doi.org/10.3390/su12093544>
- [2] A. Omeer, R. Deshmukh. (2022). “*Deep Learning-Based Models for Classification of Invasive Plant Species from Hyperspectral Remotely Sensed Data.*” Association for Computing Machinery. <https://dl.acm.org/doi/10.1145/3484824.3484884>
- [3] C. Gonçalves, P. Santana, T. Brandão, and M. Guedes. (2022). “*Automatic detection of Acacia longifolia invasive species based on UAV-acquired aerial imagery.*” Information Processing in Agriculture, Vol 9. Science Direct. <https://doi.org/10.1016/j.inpa.2021.04.007>
- [4] Crequena, T Engel, W. Kan. (2017). *Invasive Species Monitoring*. Kaggle. <https://kaggle.com/competitions/invasive-species-monitoring>
- [5] A. Roberts. (2023). *Binary Cross Entropy: Where to Use Log Loss in Model Monitoring*. Arize. <https://arize.com/blog-course/binary-cross-entropy-log-loss/#:~:text=Binary%20cross%20entropy%20>
- [6] Lamberti A. (2024). *Deep Learning Optimization Algorithms*. Neptune.Ai. neptune.ai/blog/deep-learning-optimization-algorithms
- [7] Thakur A. (2024). *What's the Optimal Batch Size to Train a Neural Network? Weights & Biases*. <https://wandb.ai/ayush-thakur/dl-question-bank/reports/What-s-the-Optimal-Batch-Size-to-Train-a-Neural-Network---VmldzoyMDkyNDU>